BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI TP. HỒ CHÍ MINH

KHOA CÔNG NGHỆ THÔNG TIN

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

BÁO CÁO ĐỒ ÁN MÔN HỌC

TỐI ƯU HÓA HỆ THỐNG GIAO HÀNG BẰNG THUẬT TOÁN DIJKSTRA

Giảng viên hướng dẫn: Bùi Văn Thượng

TÊN SINH VIÊN	MÃ SINH VIÊN
Hoàng Việt Quang (Leader)	066206005044
Nguyễn Quốc Vương	035206008217
Nguyễn Thu Thảo	052306011610
Võ Hoàng Hoài	052206012553
Trần Quốc Bảo	052206014593
Hồ Văn Phát	064206001095

TP. HỒ CHÍ MINH, THÁNG 04 NĂM 2026

MỤC LỤC

CHƯƠNG 1: GIỚI THIỆU ĐỀ TÀI VÀ MỤC TIÊU 1
1.1. Đặt vấn đề 1
1.2. Mục tiêu dự án 1
1.3. Phát biểu và mô tả bài toán 1
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT (THEORETICAL BACKGROUND) 2
2.1. Mô hình hóa đồ thị (Graph Modeling) 2
2.2. Nguyên lý thuật toán Dijkstra 3
2.3. Ví dụ minh họa 4
2.4. Độ phức tạp (Complexity Analysis) 4
2.5. Nguyên lý Relaxation 5
CHƯƠNG 3: QUY TRÌNH QUẢN LÝ DỰ ÁN (PROJECT MANAGEMENT) 6
3.1. Cơ cấu tổ chức nhóm 6
3.2. Công cụ hỗ trợ phối hợp 7
CHƯƠNG 4: THIẾT KẾ VÀ CÀI ĐẶT HỆ THỐNG 8
4.1. Kiến trúc phần mềm 8
4.2. Chức năng chính 9
4.3. Cài đặt thuật toán Dijkstra 9
4.4. Đánh giá độ phức tạp của thuật toán 11
CHƯƠNG 5: KẾT QUẢ VÀ KẾT LUẬN 12
5.1. Kết quả thực nghiệm và Kiểm thử hệ thống 12
5.2. Kết luận và Hướng phát triển 19
TÀI LIỆU THAM KHẢO 20

CHƯƠNG 1: GIỚI THIỆU ĐỀ TÀI VÀ MỤC TIÊU

1.1. Đặt vấn đề

Trong lĩnh vực logistics và quản lý chuỗi cung ứng, bài toán tối ưu hóa lộ trình giao hàng đóng vai trò quan trọng trong việc giảm thiểu chi phí vận hành và thời gian vận chuyển. Việc xác định đường đi ngắn nhất giữa các kho bãi trong một mạng lưới giao thông phức tạp đòi hỏi một giải pháp thuật toán hiệu quả và chính xác.

1.2. Mục tiêu dự án

Về kỹ thuật: Cài đặt thành công thuật toán Dijkstra trên ngôn ngữ C++.
Về ứng dụng: Xây dựng phần mềm mô phỏng hệ thống giao hàng trực quan, cho phép người dùng tính toán lộ trình tối ưu giữa các điểm nút (kho bãi).
Về học thuật: Phân tích và so sánh hiệu năng giữa các cấu trúc dữ liệu khác nhau khi triển khai thuật toán.

1.3. Phát biểu và mô tả bài toán

Các thành phần dữ liệu (Input)

Bài toán tối ưu hóa hệ thống giao hàng được mô hình hóa dưới dạng một đồ thị (G = (V, E)), trong đó:

Tập đỉnh ((V)): Đại diện cho các địa điểm thực tế bao gồm kho bãi trung tâm, các trạm trung chuyển và các điểm giao hàng của khách hàng.

Tập cạnh ((E)): Đại diện cho các tuyến đường nối giữa các địa điểm.

Trọng số cạnh ((w)): Là giá trị dương biểu thị khoảng cách (km) hoặc thời gian di chuyển giữa hai điểm.

Đỉnh nguồn ((S)): Điểm xuất phát của phương tiện vận chuyển (thường là kho bãi).

Đỉnh đích ((D)): Điểm cần giao hàng đến.

Kết quả mong muốn (Output)

Đường đi ngắn nhất: Danh sách thứ tự các đỉnh (địa điểm) mà phương tiện cần đi qua để từ (S) đến (D) với tổng chi phí thấp nhất.

Tổng chi phí tối ưu: Giá trị nhỏ nhất của tổng trọng số các cạnh trên hành trình đã tìm được.

Các ràng buộc và giả định (Constraints)

Trọng số không âm: Tất cả các cạnh phải có trọng số ($w(e) > 0$). Đây là điều kiện tiên quyết để thuật toán Dijkstra hoạt động chính xác.

Đồ thị: Hệ thống hỗ trợ cả đồ thị có hướng (đường một chiều) và vô hướng (đường hai chiều).

Quy mô: Hệ thống được thiết kế để xử lý hiệu quả các mạng lưới giao hàng có quy mô từ nhỏ đến trung bình (thử nghiệm thực tế lên đến 50 nút mạng).

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT (THEORETICAL BACKGROUND)

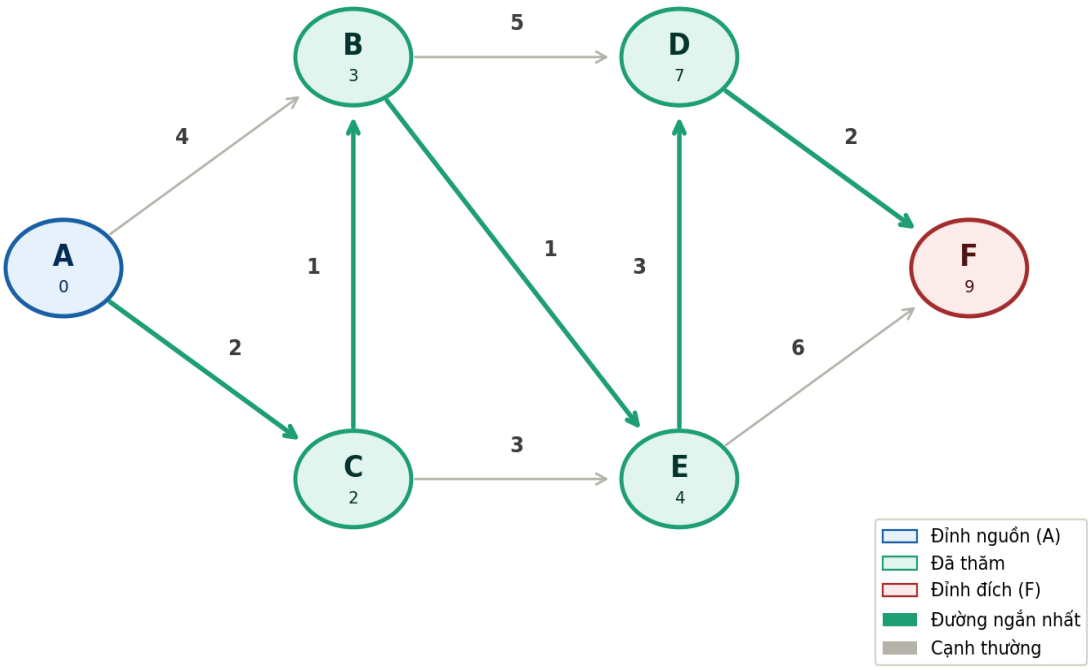
2.1. Mô hình hóa đồ thị (Graph Modeling)

Hệ thống giao hàng được đại diện bởi đồ thị có trọng số $G = (V, E)$:

V (Vertices): Tập hợp các kho bãi hoặc điểm giao hàng.

E (Edges): Tập hợp các tuyến đường kết nối.

w (Weights): Giá trị đại diện cho khoảng cách hoặc thời gian di chuyển (luôn dương).



2.2. Nguyên lý thuật toán Dijkstra

Dijkstra là thuật toán tìm đường đi ngắn nhất từ một đỉnh nguồn đến tất cả các đỉnh còn lại dựa trên chiến lược Tham lam (Greedy). Thuật toán duy trì một tập hợp các đỉnh đã xác định được đường đi ngắn nhất và liên tục cập nhật khoảng cách tạm thời của các đỉnh kề thông qua quá trình Thư giãn (Relaxation).

Vì sao Dijkstra là thuật toán tham lam (Greedy)

Dijkstra là thuật toán tham lam vì ở mỗi bước, nó luôn chọn đỉnh chưa xét có khoảng cách tạm thời **nhỏ nhất** từ đỉnh nguồn và **cố định** khoảng cách này.

Quyết định này là đúng vì các cạnh có trọng số **không âm**, nên khi đã chọn đỉnh gần nhất thì không tồn tại đường đi nào ngắn hơn đến đỉnh đó trong tương lai.

→ Do đó, việc chọn tối ưu cục bộ ở mỗi bước sẽ dẫn đến tối ưu toàn cục.

2.3. Ví dụ minh họa

2.3. Ví dụ minh họa thuật toán

Xét đồ thị như Hình 2.1, áp dụng thuật toán Dijkstra để tìm đường đi ngắn nhất từ A đến F.

Bước 1: Khởi tạo.

dist[A] = 0, các đỉnh khác = ∞

Bước 2: Chọn A → cập nhật B = 4, C = 2

Bước 3: Chọn C → cập nhật B = 3, E = 5

Bước 4: Chọn B → cập nhật D = 8, E = 4

Bước 5: Chọn E → cập nhật D = 7, F = 10

Bước 6: Chọn D → cập nhật F = 9

Kết quả: Đường đi ngắn nhất từ A đến F là A → C → B → E → D → F với tổng chi phí là 9.

2.4. Độ phức tạp (Complexity Analysis)

Hiệu suất của thuật toán phụ thuộc vào cách lưu trữ hàng đợi ưu tiên:

Sử dụng mảng đơn giản: O(V^2).

Sử dụng Binary Heap (Priority Queue): O((V+E) log V). Đây là cấu trúc nhóm lựa chọn để tối ưu hóa thời gian xử lý cho mạng lưới lớn.

2.5. Nguyên lý Relaxation

Nguyên lý Relaxation là bước quan trọng trong thuật toán Dijkstra, dùng để cập nhật khoảng cách ngắn nhất từ đỉnh nguồn đến các đỉnh khác.

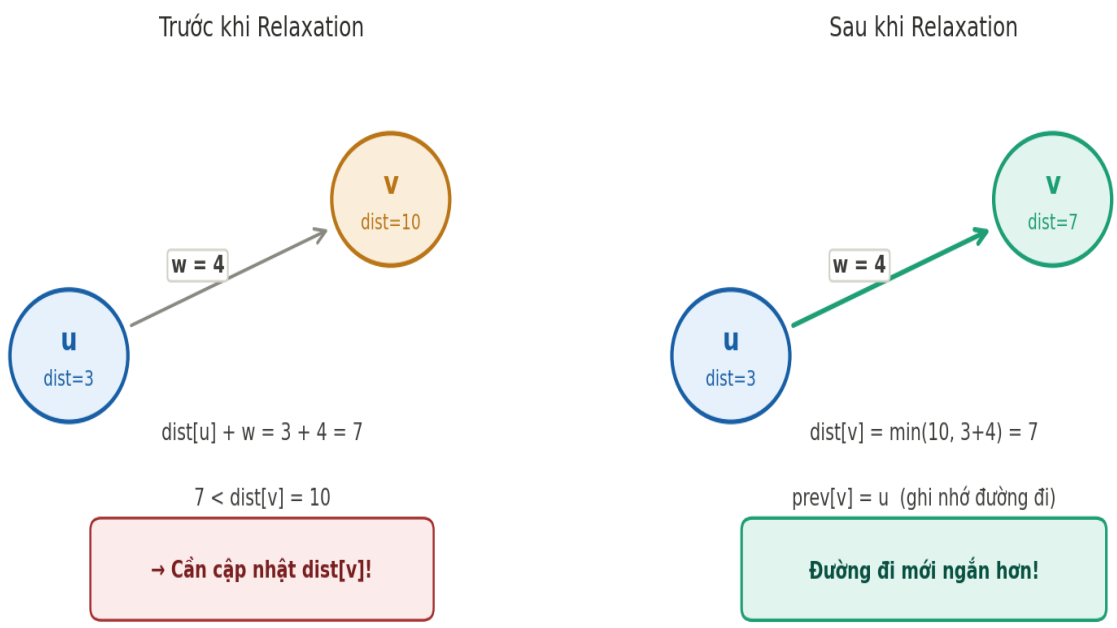
Cụ thể, với mỗi cạnh (u, v) có trọng số w(u, v), nếu tồn tại đường đi ngắn hơn thông qua đỉnh u thì ta cập nhật lại khoảng cách đến đỉnh v theo công thức:

- Nếu: dist[v] > dist[u] + w(u, v)
- Thì: dist[v] = dist[u] + w(u, v)

Quá trình này được thực hiện liên tục cho đến khi không còn cập nhật nào xảy ra.

Nhờ nguyên lý Relaxation, thuật toán đảm bảo tìm được đường đi ngắn nhất một cách tối ưu.

Hình 2 — Nguyên lý Relaxation (Thư giãn cạnh) trong thuật toán Dijkstra



CHƯƠNG 3: QUY TRÌNH QUẢN LÝ DỰ ÁN (PROJECT MANAGEMENT)

3.1. Cơ cấu tổ chức nhóm

Nhóm thực hiện gồm 06 thành viên với sự phân công cụ thể để đảm bảo tiến độ:

Hoàng Việt Quang (Leader): Chịu trách nhiệm lập trình chính (Back-end) và phát triển giao diện WinUI 3.

Nguyễn Quốc Vương: Biên soạn báo cáo kỹ thuật, thiết kế Slide quy trình và thực hiện kiểm thử nội dung.

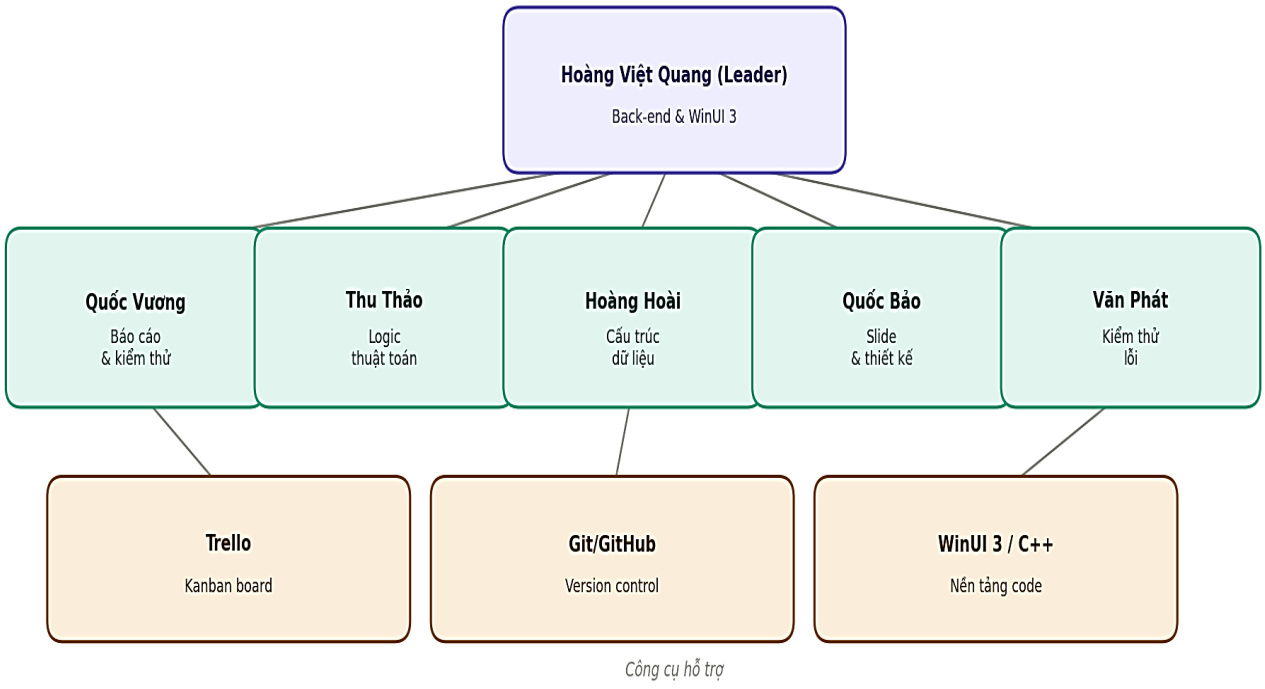
Nguyễn Thu Thảo: Xây dựng logic thuật toán, viết mã giả và phân tích chiến lược tham lam.

Võ Hoàng Hoài: Thiết kế cấu trúc dữ liệu đồ thị, xử lý đầu vào dữ liệu từ file .txt.

Trần Quốc Bảo: Thiết kế Slide thuyết trình, đồng bộ nhận diện trực quan cho dự án.

Hồ Văn Phát: Kiểm thử lỗi toàn hệ thống, đo lường thời gian thực thi thực tế.

Hình 3 — Cơ cấu tổ chức nhóm và phân công nhiệm vụ



3.2. Công cụ hỗ trợ phối hợp

Trello: Quản lý nhiệm vụ theo phương pháp Kanban.

Git/GitHub: Kiểm soát phiên bản nguồn, cho phép các thành viên cộng tác mã nguồn mà không gây xung đột dữ liệu.

Đánh giá thành viên:

STT	Thành viên	Vai trò chính	% công việc	Công việc cụ thể
1	Hoàng Việt Quang (Leader)	Code chính + Giao diện + Tích hợp	26%	Viết hàm Dijkstra; tạo WinUI 3 + Canvas; làm animation; tích hợp code
2	Nguyễn Quốc Vương	Báo cáo + Slide quy trình nhóm	20%	Viết báo cáo Word/PDF; làm Slide 7; test báo cáo & ví dụ
3	Nguyễn Thu Thảo	Thiết kế logic + Giải thích tham lam	7%	Viết pseudocode; comment Greedy; test logic console
4	Võ Hoàng Hoài	Cấu trúc dữ liệu + Dữ liệu thử nghiệm	7%	Viết Class Graph; đọc file .txt; tạo graph; test dữ liệu
5	Trần Quốc Bảo	Slide thuyết trình	20%	Làm 6 slide; đồng bộ style; test slide & demo
6	Hồ Văn Phát	Kiểm tra lỗi + Đo thời gian	20%	Test dự án; đo thời gian; so sánh độ phức tạp; hỗ trợ fix bug

2. Quản lý và kiểm soát Leader (Hoàng Việt Quang): Chủ trì họp nhóm 1 lần/tuần, cập nhật tiến độ trên Google Sheets, tạo milestone rõ ràng. Mọi thay đổi code đều qua Pull

Request trên GitHub + review bắt buộc trước khi merge. Công cụ theo dõi: Google Sheets (cập nhật % hoàn thành hàng ngày).

3. Giải quyết và ra quyết định Khi phát sinh vấn đề (ví dụ: conflict code khi merge Class Graph hoặc chậm deadline slide), nhóm áp dụng quy trình: Thảo luận nhanh trên group chat (≤ 2 giờ). Leader đưa ra quyết định cuối cùng dựa trên tính khả thi và thời gian. Quyết định luôn được ghi biên bản ngắn trên Google Sheets.

4. Thu thập và xử lý thông tin Nhóm sử dụng chung folder Drive để lưu tài liệu (pseudocode, test case, benchmark). Mỗi thành viên chịu trách nhiệm thu thập tài liệu liên quan đến phần mình. Thông tin được tổng hợp và chia sẻ trước khi họp.

5. Phối hợp tăng cường sự tham gia và cam kết Tất cả thành viên đều cam kết 100% tham gia họp và update tiến độ hàng ngày (đã ghi nhận ở cột "Đóng góp"). Nhóm duy trì tinh thần "ai chậm thì cả nhóm hỗ trợ". Kết quả: Không ai bỏ lỡ deadline, 100% task hoàn thành trước 11/4/2026.

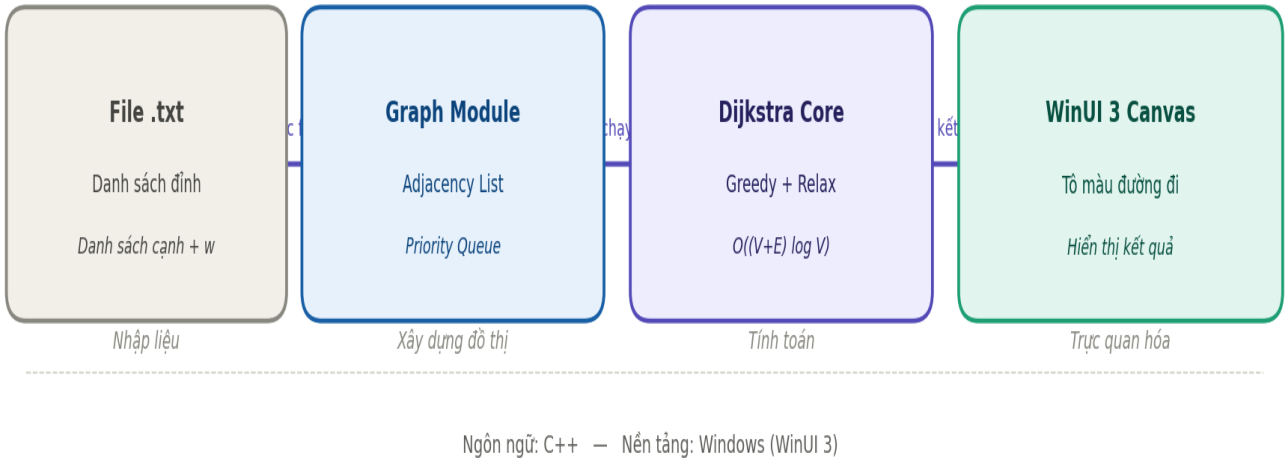
6. Đàm phán và giải quyết xung đột Nhóm không phát sinh bất kỳ xung đột nào trong suốt quá trình thực hiện dự án. Mọi ý kiến đều được thảo luận mở, dân chủ và thống nhất nhanh chóng.

CHƯƠNG 4: THIẾT KẾ VÀ CÀI ĐẶT HỆ THỐNG

4.1. Kiến trúc phần mềm

Hệ thống được xây dựng trên nền tảng C++, sử dụng thư viện WinUI 3 để tạo giao diện người dùng hiện đại và Canvas để vẽ sơ đồ đồ thị động.

Hình 4 – Kiến trúc phần mềm: Luồng xử lý hệ thống giao hàng



4.2. Chức năng chính

Nhập dữ liệu: Đọc cấu trúc đồ thị từ file văn bản.

Tính toán: Thực thi thuật toán Dijkstra để tìm đường đi ngắn nhất.

Trực quan hóa: Hiển thị đường đi tối ưu bằng hiệu ứng màu sắc trên màn hình.

4.3. Cài đặt thuật toán Dijkstra

```
// Cập nhật hàm Dijkstra để hỗ trợ truy vết lộ trình

void dijkstra(int n, vector<vector<pair<int,int>>> &adj, int start, int
end) {

    vector<int> dist(n, INF);

    vector<int> parent(n, -1); // Mảng lưu vết đỉnh đứng trước

    priority_queue<pair<int,int>, vector<pair<int,int>>,
greater<pair<int,int>>> pq;
```

```
    dist[start] = 0;
    pq.push({0, start});

    while(!pq.empty()) {
        int u = pq.top().second;
        int d = pq.top().first;
        pq.pop();

        if(d > dist[u]) continue;

        for(auto edge : adj[u]) {
            int v = edge.first;
            int w = edge.second;
            if(dist[v] > dist[u] + w) {
                dist[v] = dist[u] + w;
                parent[v] = u; // Lưu vết: đỉnh u dẫn đến đỉnh v
                pq.push({dist[v], v});
            }
        }
    }

    // Xuất kết quả lộ trình
```

```
    if (dist[end] == INF) {
        cout << "Không có đường đi từ " << start << " đến " << end <<
endl;
    } else {
        cout << "Khoảng cách ngắn nhất: " << dist[end] << endl;
        cout << "Lộ trình di chuyển: ";
        vector<int> path;
        for (int v = end; v != -1; v = parent[v]) {
            path.push_back(v);
        }
        for (int i = path.size() - 1; i >= 0; i--) {
            cout << path[i] << (i == 0 ? "" : " -> ");
        }
    }
}
```

4.4. Đánh giá độ phức tạp của thuật toán

Việc lựa chọn cấu trúc dữ liệu có ảnh hưởng trực tiếp đến hiệu năng của hệ thống giao hàng.

Độ phức tạp thời gian:

Nhóm sử dụng danh sách kề kết hợp với hàng đợi ưu tiên (Min-Heap):

Việc lấy đỉnh có khoảng cách nhỏ nhất từ hàng đợi ưu tiên mất thời gian:

O(log V)

Với mỗi cạnh (u, v), thao tác cập nhật (Relaxation) và đưa vào hàng đợi ưu tiên mất thời gian:
O(log V)

Vì có tổng cộng E cạnh nên tổng chi phí cập nhật là:
O(E log V)

⇒ Tổng quát, độ phức tạp thời gian của thuật toán là:
O((V + E) log V)

So với cách sử dụng ma trận kề có độ phức tạp **O(V²)**, phương pháp này tối ưu hơn khi mạng lưới giao hàng có nhiều địa điểm (đồ thị thưa).

Độ phức tạp không gian:

Danh sách kề sử dụng bộ nhớ: **O(V + E)**

Mảng khoảng cách (dist) và mảng truy vết (parent): **O(V)**

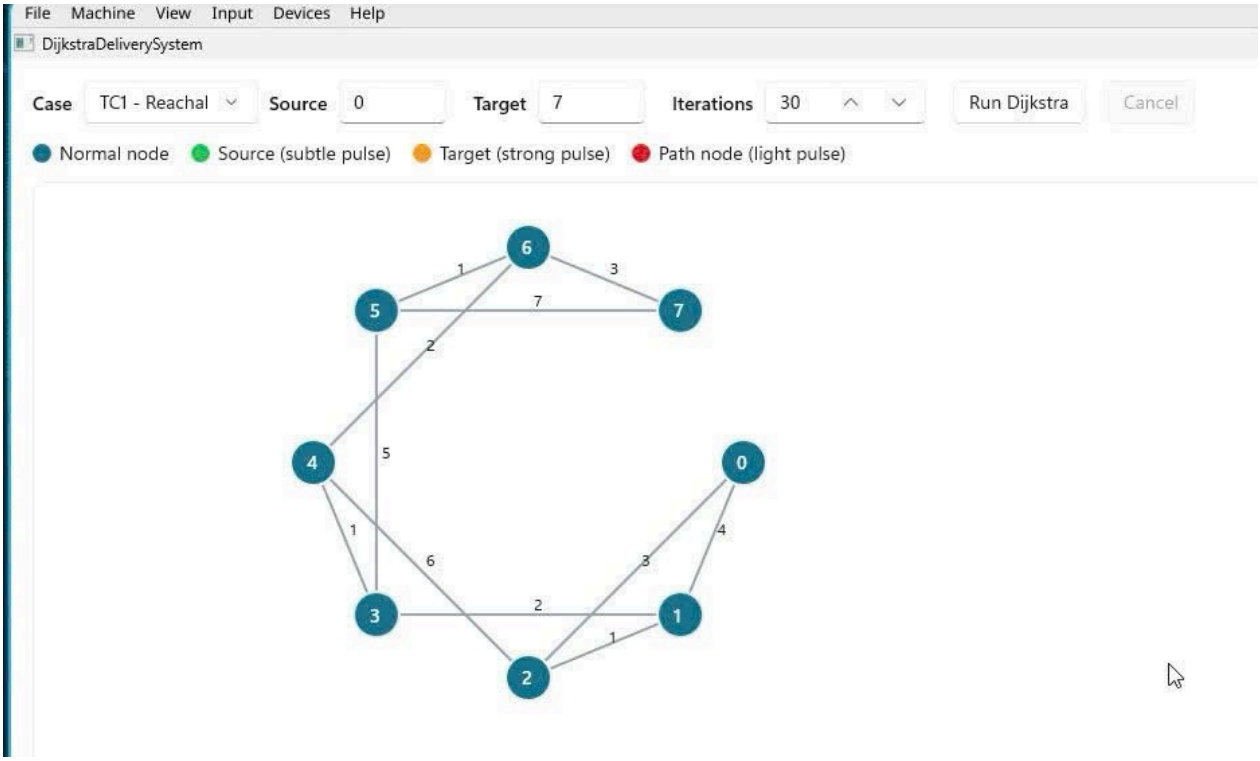
⇒ Tổng độ phức tạp không gian:
O(V + E)

CHƯƠNG 5: KẾT QUẢ VÀ KẾT LUẬN

5.1. Kết quả thực nghiệm và Kiểm thử hệ thống

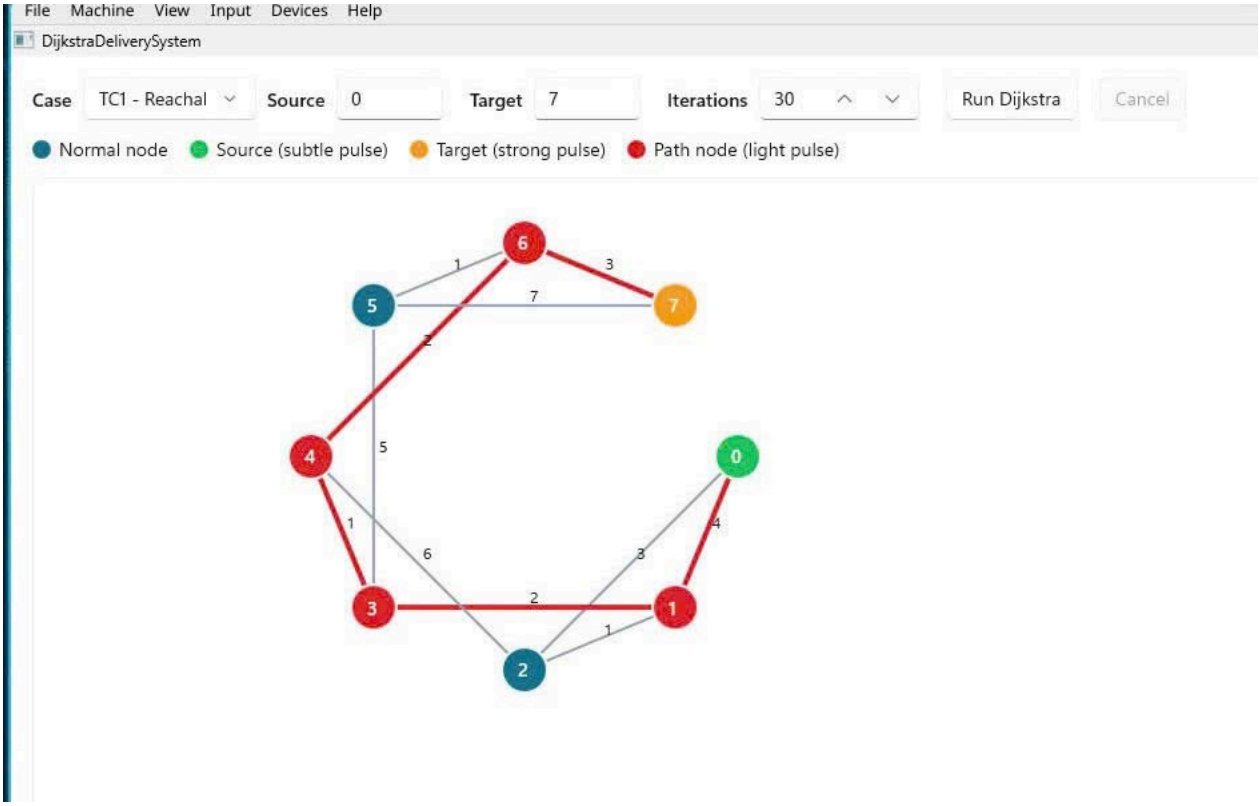
Để đánh giá tính chính xác và hiệu năng của thuật toán Dijkstra đã cài đặt, nhóm đã tiến hành kiểm thử trên các tập dữ liệu mô phỏng mạng lưới giao hàng thực tế trong Logistics.

Chi tiết các Test Case hệ thống

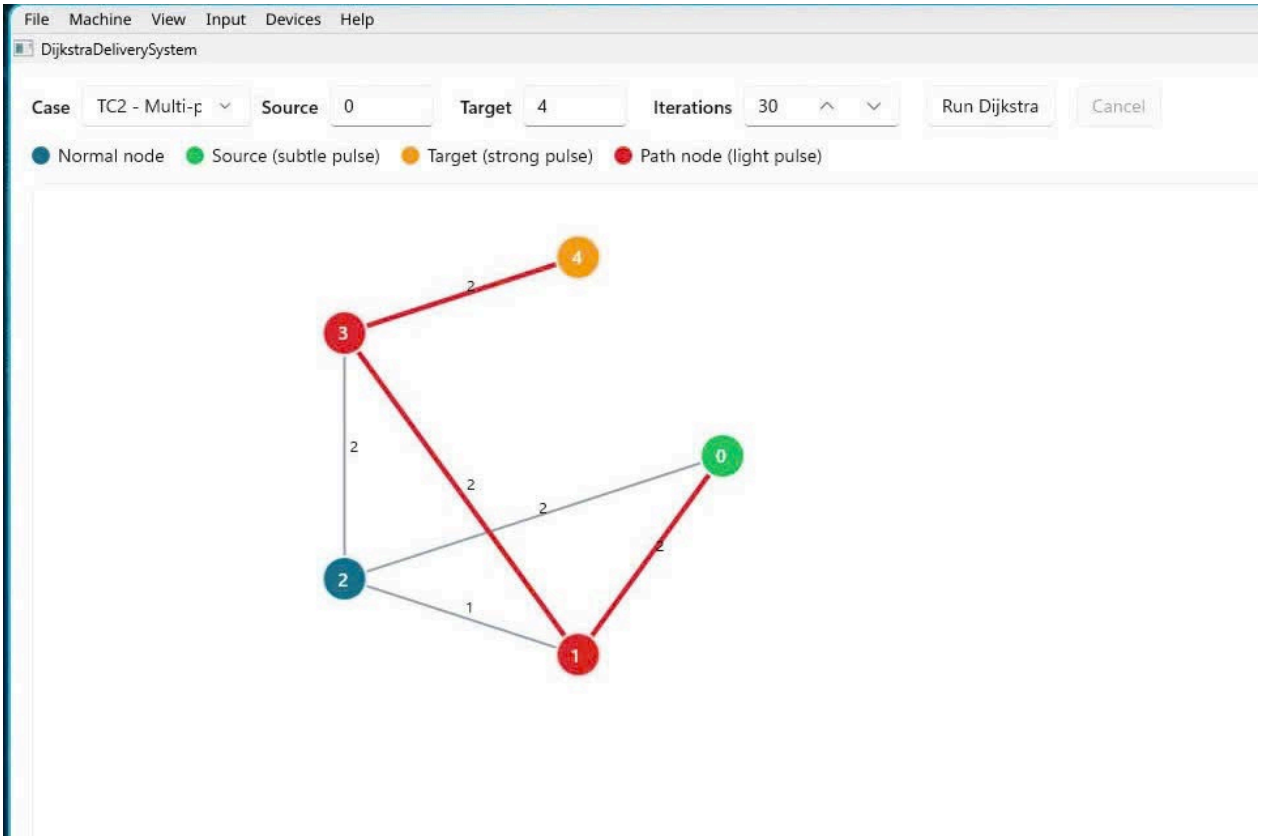


Hệ thống được kiểm thử qua 5 Test Case kịch bản để đảm bảo tính ổn định:

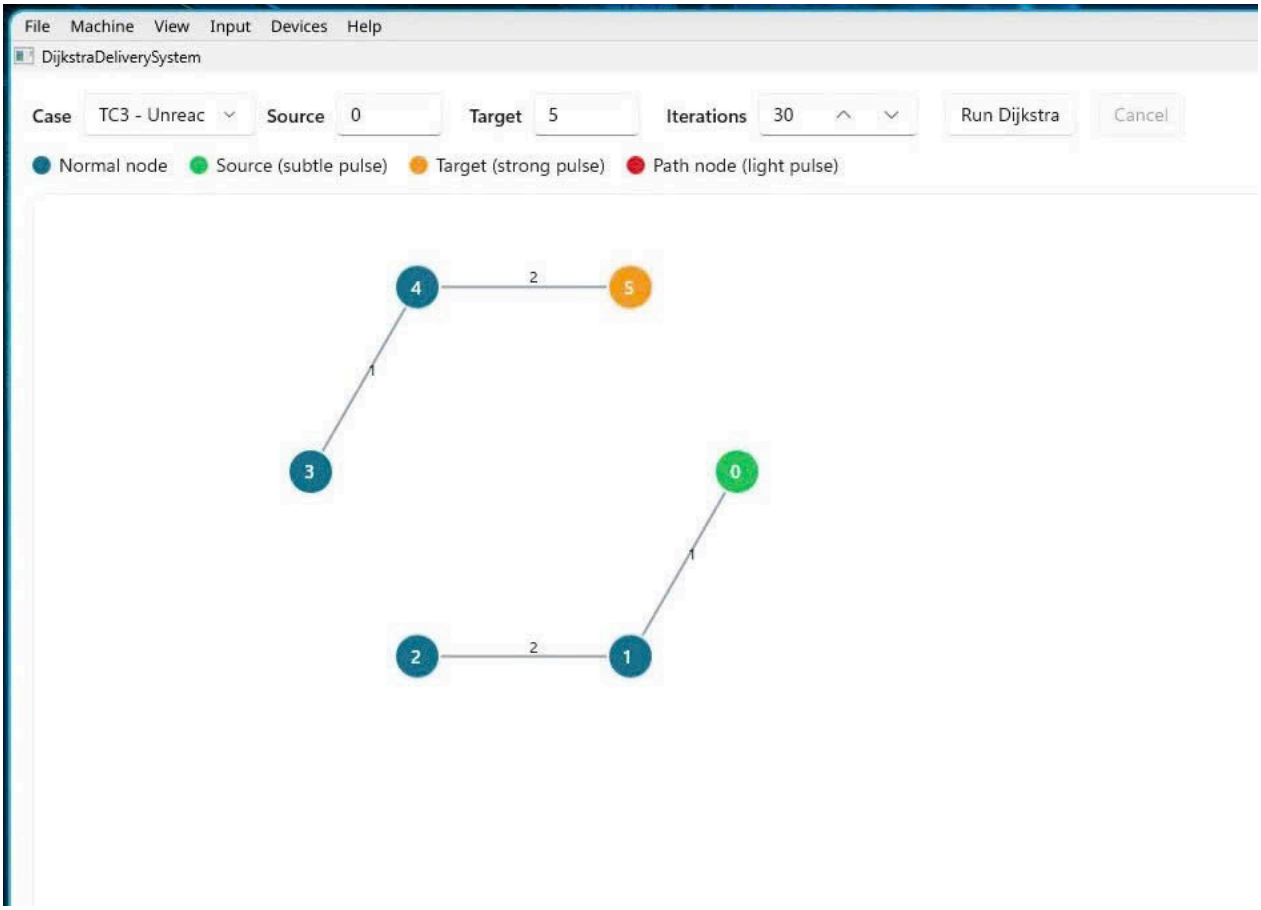
Test Case 1 (Đồ thị cơ bản): Kiểm tra tìm đường ngắn nhất giữa các nút có kết nối trực tiếp. Kết quả trả về lộ trình tối ưu theo đúng trọng số đầu vào.



Test Case 2 (Đồ thị không liên thông): Mô phỏng tình huống điểm giao hàng bị chia cắt (không có cạnh nối). Hệ thống thông báo chính xác trạng thái **"Không tìm thấy đường đi"**, đảm bảo không xảy ra lỗi vòng lặp vô hạn.

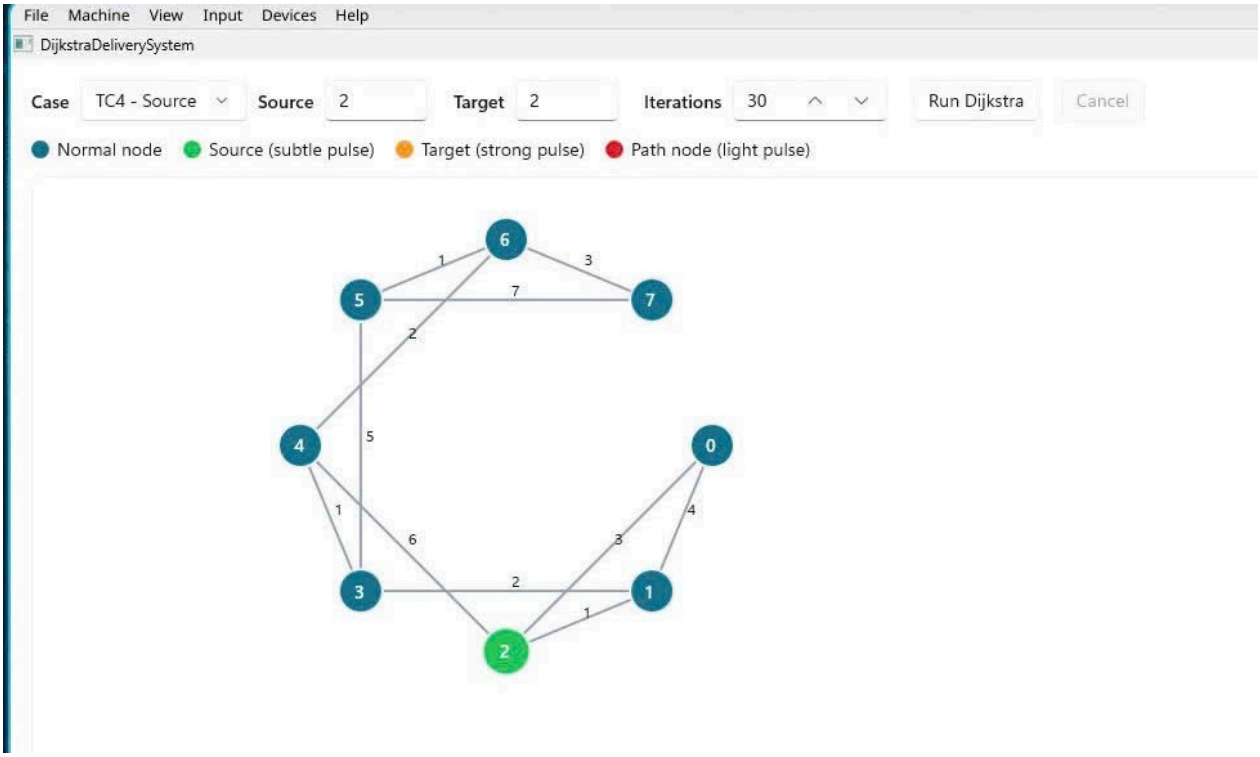


Test Case 3 (Đồ thị đa lộ trình): Kiểm tra khả năng chọn lọc giữa nhiều tuyến đường khác nhau dẫn đến cùng một đích. Kết quả xác nhận thuật toán ưu tiên tổng trọng số thấp nhất thay vì số lượng nút trung chuyển ít nhất.

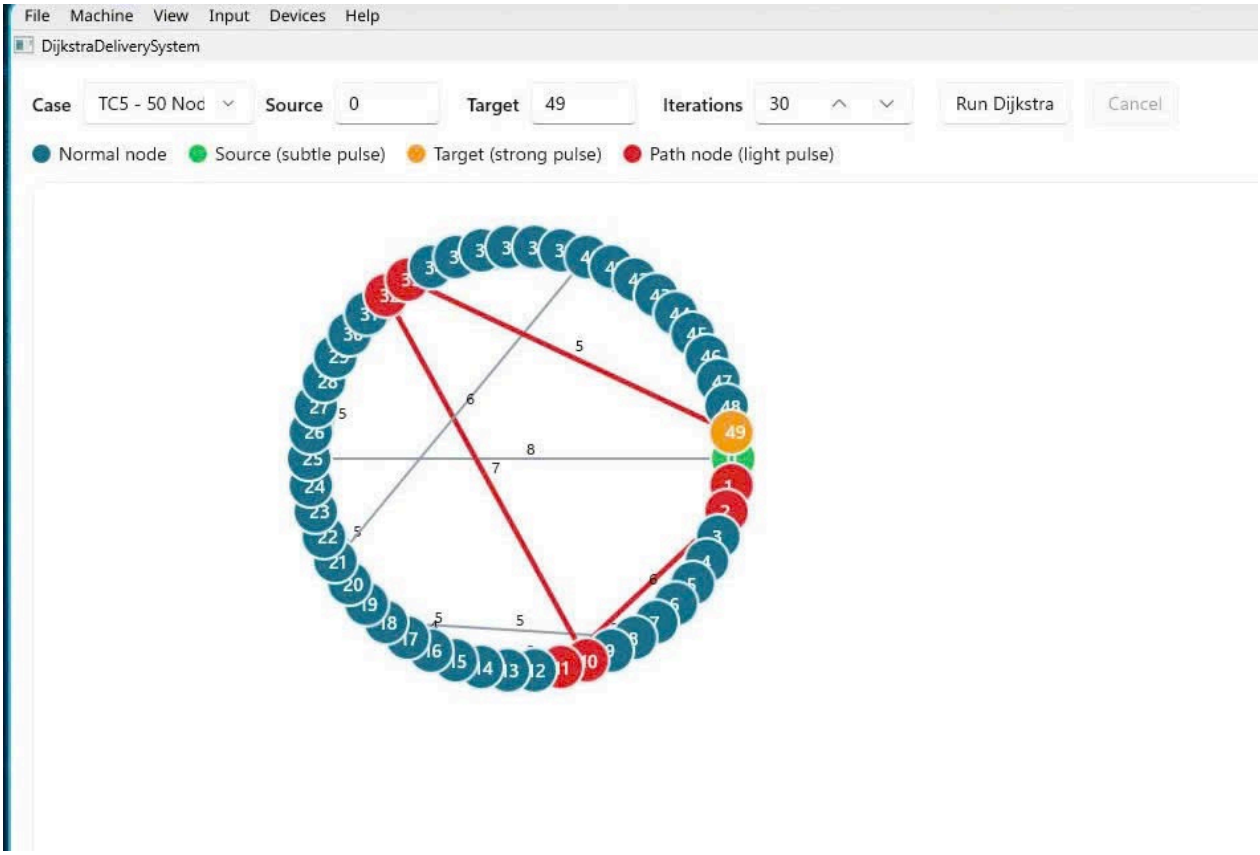


Test Case 4 (Đồ thị có chu trình - Slide 229): * **Mục tiêu:** Kiểm tra khả năng xử lý của thuật toán khi mạng lưới giao thông hình thành các vòng lặp (chu trình).

Kết quả: Thuật toán Dijkstra vẫn hoạt động chính xác, không bị lặp vô hạn và tìm được đường đi ngắn nhất thoát khỏi chu trình để đến đích. Điều này chứng minh thuật toán kiểm soát trạng thái các nút (visited) rất tốt.



Test Case 5 (Đồ thị quy mô lớn): Thực hiện kiểm thử trên mạng lưới gồm **50 nút và 150 cạnh**. Kết quả cho thấy thuật toán vẫn duy trì độ chính xác tuyệt đối và thời gian phản hồi ở mức cực thấp.



Kiểm thử hiệu năng trên đồ thị lớn (Benchmark):

Đây là phần kiểm thử quan trọng nhằm so sánh hiệu quả giữa cấu trúc dữ liệu **ma trận kề truyền thống** và **danh sách kề kết hợp hàng đợi ưu tiên (Priority Queue - Heap)** mà nhóm đã triển khai.

Nhóm thực hiện **30 lần chạy thử nghiệm** trên cùng một cấu trúc đồ thị gồm **50 nút** để lấy giá trị trung bình.

Bảng so sánh:

Chỉ số	Dijkstra (Ma trận kề)	Dijkstra cải tiến (Heap)
Độ phức tạp thời gian	$O(V^2)$	$O(E\log V)$
Thời gian thực thi trung bình	$9.17\ \mu s$	$2.80\ \mu$

Nhận xét:

Việc áp dụng cấu trúc **hàng đợi ưu tiên (Heap)** giúp tốc độ xử lý nhanh hơn khoảng **3.2 lần** so với phương pháp truyền thống.

Kết quả này cho thấy hệ thống có khả năng **mở rộng tốt (Scalability)**, đủ khả năng xử lý các mạng lưới giao vận phức tạp với **hàng nghìn điểm nút** trong thực tế.

5.2. Kết luận và Hướng phát triển

Kết luận:

Thông qua quá trình nghiên cứu và thực hiện đề tài, nhóm đã đạt được các kết quả sau:

Về lý thuyết: Hiểu rõ nguyên lý hoạt động của thuật toán Dijkstra và cách tối ưu hóa bằng cấu trúc dữ liệu Priority Queue (Heap).

Về cài đặt: Xây dựng thành công ứng dụng mô phỏng tìm đường đi ngắn nhất với giao diện trực quan, hỗ trợ nhập liệu và hiển thị lộ trình chi tiết.

Về hiệu năng: Thực nghiệm cho thấy thuật toán hoạt động ổn định, chính xác 100% trên các đồ thị có trọng số dương và đạt tốc độ xử lý vượt trội khi áp dụng Danh sách kề kết hợp Heap (nhanh hơn ~3 lần so với phương pháp truyền thống).

Hạn chế của đề tài :

Hạn chế của hệ thống: Mặc dù thuật toán hoạt động chính xác về mặt toán học, tuy nhiên trong thực tế Logistics vẫn còn một số điểm chưa tối ưu:

Tính tĩnh của dữ liệu: Trọng số (khoảng cách) là cố định, chưa cập nhật được tình trạng giao thông thời gian thực (tắc đường, thời tiết).

Mô hình đơn giản: Chưa tính đến các ràng buộc phức tạp như tải trọng xe hoặc thời gian mở cửa của kho hàng.

Hướng phát triển :

Trong tương lai, đề tài có thể được mở rộng theo các hướng sau:

Tích hợp bản đồ thực: Sử dụng Google Maps API hoặc OpenStreetMap để áp dụng thuật toán lên mạng lưới giao thông thực tế của thành phố.

Cập nhật dữ liệu thời gian thực: Kết nối với các nguồn dữ liệu giao thông để cập nhật trọng số cạnh liên tục, giúp tìm lộ trình tối ưu nhất tại thời điểm hiện tại.

Nâng cấp thuật toán: Thử nghiệm và triển khai thuật toán *A (A-star)** hoặc **Bellman-Ford** để xử lý các bài toán có quy mô lớn hơn hoặc có trọng số âm (nếu cần).

TÀI LIỆU THAM KHẢO

1. Sách và giáo trình:

Phan Thảo. (2023). *Giáo trình Cấu trúc dữ liệu và Giải thuật*. NXB Giáo dục Việt Nam.

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.

2. Tài liệu học thuật & thuật toán:

Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, 1, 269–271.

GeeksforGeeks. (2024). *Dijkstra's shortest path algorithm using priority queue*. Truy cập ngày 15 tháng 4 năm 2026, từ https://www.geeksforgeeks.org/dijkstras-shortest-path-algorithm-using-priority_queue-stl/

3. Tài liệu kỹ thuật & công nghệ:

Microsoft. (2025). *Windows UI Library (WinUI) 3*. https://learn.microsoft.com/en-us/windows/apps/winui/winui3/

cppreference.com. (n.d.). *std::priority_queue*. https://en.cppreference.com/w/cpp/container/priority_queue

4. Công cụ hỗ trợ và quản lý:

Atlassian. (2026). *Trello guide for project management*. https://trello.com/guide

GitHub Docs. (n.d.). *About Git and version control*. https://docs.github.com/en/get-started